Threads Java, concurrence et exécution parallèle sur la JVM

Notes d'informatique et de génie logiciel

November 2, 2026

1 Fondamentaux de la concurrence et des threads sur la JVM

1.1 Concurrence vs parallélisme

En Java, la **concurrence** et le **parallélisme** sont deux concepts distincts, gérés à la fois au niveau du langage et de la machine virtuelle Java (JVM) :

- **Concurrence** : Composition de processus ou de tâches s'exécutant indépendamment. Elle concerne la *structure* de l'exécution. Un système est concurrent lorsqu'il peut gérer plusieurs tâches en faisant se chevaucher leurs périodes d'exécution, souvent sur un seul cœur de processeur grâce au partage du temps d'exécution (commutation de contexte gérée par l'ordonnanceur du système d'exploitation).
- **Parallélisme** : Exécution simultanée de plusieurs calculs. Il concerne l'*exécution*. Le parallélisme nécessite des capacités matérielles physiques (plusieurs cœurs ou processeurs) permettant à deux threads JVM ou plus d'exécuter des instructions exactement au même instant.

La JVM délègue l'ordonnancement des threads au système d'exploitation hôte (modèle de threads natifs). Par conséquent, la possibilité d'exécuter des tâches Java simultanément dépend directement des ressources matérielles disponibles.

1.2 Analogie : les threads comme des chefs cuisiniers

Pour visualiser la manière dont la JVM gère les unités d'exécution :

- **La cuisine (CPU / cœurs)** : Représente l'environnement matériel d'exécution.
- **Les recettes (tâches / `Runnable`)** : Représentent le code ou les instructions à exécuter.
- **Les chefs (threads)** : Représentent les contextes d'exécution actifs qui réalisent les recettes.

  Un seul cœur (un chef) : Un chef peut travailler sur plusieurs plats en même temps (couper des oignons pendant que l'eau chauffe). Il passe d'une tâche à l'autre (concurrence), mais à un instant précis, une seule action est physiquement effectuée.

  Plusieurs cœurs (plusieurs chefs) : Deux chefs travaillent côte à côte sur des postes distincts. L'un fait cuire du pain pendant que l'autre fait saisir un steak simultanément (parallélisme).

1.3 Cœurs CPU vs threads (threads du système d'exploitation vs threads JVM)

- **Cœurs matériels** : Unités physiques de traitement intégrées au processeur. Un processeur à 4 cœurs avec l'hyper-threading peut exécuter jusqu'à 8 threads matériels simultanément.
- **Threads du système d'exploitation** : Threads de niveau noyau gérés par le système d'exploitation. Ils entraînent des coûts liés aux changements de contexte (sauvegarde des registres, mise à jour des pointeurs de pile et invalidation du cache).
- **Threads de plateforme (threads Java traditionnels)** : Avant Java 21, chaque instance de `java.lang.Thread` était une enveloppe légère autour d'un thread natif du système d'exploitation (correspondance 1 : 1). Leur création est coûteuse (environ 1 Mo d'allocation de pile par thread), ce qui limite le nombre total de threads actifs qu'une JVM peut maintenir (généralement quelques milliers).

1.4 Mémoire Heap vs Stack dans le contexte des threads

La JVM divise la mémoire en régions partagées et isolées :

Le Heap Java (mémoire partagée) Tous les threads partagent un même Heap. Les objets, les instances de classes et les tableaux y résident. Comme tous les threads peuvent lire et modifier simultanément les données du Heap, les états mutables nécessitent des mécanismes de synchronisation (tels que `synchronized`, les verrous ou les variables atomiques) afin d'éviter les conditions de concurrence critiques (race conditions).

La pile des threads (mémoire isolée) Chaque thread possède sa propre pile d'appels privée. Les variables locales primitives et les pointeurs de référence vers les objets situés dans le Heap sont stockés dans la pile du thread. Aucun thread ne peut accéder directement à la pile d'un autre thread, ce qui rend les variables locales intrinsèquement sûres du point de vue de la concurrence.

1.5 Cas d'utilisation pratiques : pourquoi les threads et les tâches sont importants

Les architectures logicielles réelles s'appuient sur les threads pour assurer la réactivité et le débit de traitement :

1. **Réactivité des interfaces utilisateur** : Les interfaces utilisateur (Swing, JavaFX, Android) utilisent un thread dédié appelé Event Dispatch Thread (EDT). Déplacer les tâches de calcul intensives ou les opérations d'entrée-sortie (I/O) vers des threads de travail évite de bloquer l'EDT et permet à l'application de rester interactive.
2. **Traitement en arrière-plan (threads démons)** : Les tâches de longue durée, telles que l'agrégation des journaux, le ramasse-miettes (garbage collection) et la synchronisation de la télémétrie, s'exécutent sur des threads démons en arrière-plan, sans empêcher la terminaison du cycle de vie de l'application.
3. **Tâches planifiées** : Les tâches récurrentes, comme la récupération des cours boursiers toutes les 5 secondes ou la génération de sauvegardes quotidiennes de bases de données, reposent sur des threads de planification dédiés afin de s'exécuter périodiquement sans nécessiter de boucles de vérification continues dans le programme principal.

2 Abstractions modernes des threads Java et modèles d'exécution

2.1 Interface `Runnable`

L'interface `Runnable` représente une unité de travail (une tâche) destinée à être exécutée par un thread. Elle permet de séparer la *définition de la tâche* de son *mécanisme d'exécution.*

2.2 Pools de threads (`ExecutorService`)

La création manuelle de threads de plateforme adossés au système d'exploitation est coûteuse en ressources. Les pools de threads gèrent un ensemble de threads de travail réutilisables :

- Réduisent la surcharge en évitant la création et la destruction fréquentes de threads.
- Contrôlent l'utilisation des ressources en plaçant les tâches excédentaires dans une file d'attente lorsque tous les threads du pool sont occupés.
- Sont gérés à l'aide de méthodes de fabrique telles que `Executors.newFixedThreadPool(int n)` ou `Executors.newWorkStealingPool()`.

2.3 Classes anonymes vs expressions lambda en Java

Java a évolué afin de réduire la verbosité lors de la définition des tâches :

- **Classes anonymes** : Instancient une classe en ligne implémentant `Runnable`. Le compilateur génère un fichier `.class` distinct et crée une instance d'objet spécifique possédant son propre contexte `this`.
- **Expressions lambda (Java 8+)** : Fournissent une syntaxe fonctionnelle plus concise. Elles utilisent l'instruction JVM `invokedynamic`, évitant ainsi la génération inutile de fichiers de classes et partageant le contexte de la portée englobante.

2.4 Threads virtuels (Project Loom / Java 21+)

Les threads virtuels introduisent une alternative légère aux threads de plateforme traditionnels :

- **Modèle d'ordonnancement** $M : N$: Des millions de threads virtuels (M) sont multiplexés sur un petit ensemble de threads porteurs du système d'exploitation (carrier threads) (N).
- **Blocage sans immobilisation du thread porteur** : Lorsqu'un thread virtuel rencontre une opération d'entrée-sortie bloquante (par exemple, l'attente d'une requête vers une base de données), la JVM détache le thread virtuel de son thread porteur du système d'exploitation, libérant ainsi ce dernier pour exécuter d'autres threads virtuels. Lorsque l'opération d'entrée-sortie est terminée, le thread virtuel est rattaché à un thread porteur disponible.
- **Empreinte mémoire** : Les besoins en mémoire passent de plusieurs mégaoctets à quelques centaines d'octets par thread, permettant ainsi au débit de traitement de s'adapter au nombre de connexions réseau plutôt que d'être limité directement par le nombre de cœurs matériels.